

Pipeline AI CRM

Two suites were run against the application: an automated suite covering every API route and the scoring, query, matching and safety logic, and an end-to-end pass that drives the real interface in a browser and photographs each state. 192 of 192 checks passed.

147 API & UNIT TESTS	45 UI CHECKS	44 ROUTES COVERED	41 SCREENS CAPTURED	0 FAILING
-------------------------	-----------------	----------------------	------------------------	--------------

What was tested

Breadth first: every route with its auth, validation and role rules. Then depth on the logic that would be expensive to get wrong, and finally the interface itself, driven as a person would use it.

Automated suite — 147 tests across 9 files

FILE	TESTS	COVERS
api.full.test.ts	86	Every route on every router, with auth, validation, role and success paths
nlQuery.test.ts	15	Natural-language query validation, rejection, compilation and the date grammar
duplicates.test.ts	14	Fuzzy matching, nickname and typo handling, and blocking-key recall
leadScore.test.ts	9	Each scoring component, bounds, and the change-detection hash
contacts.crud.test.ts	7	The core CRUD journey with role scoping and background scoring
riskFlag.test.ts	5	Each risk signal and the reasons it produces
sanitize.test.ts	5	Prompt-injection hardening of untrusted text
aiGateway.test.ts	3	Caching, usage logging, circuit breaker, refusals and timeouts
semanticSearch.test.ts	3	Meaning-based ranking, ownership scoping and the text fallback

Endpoint coverage by router

ROUTER	TESTS
Health and 404	3
Auth	8
Contacts	14
Deals	16
Notes	8
Tasks	8
Meetings	4
Ai	6
Duplicates	5
Dashboard	2
Admin	5
Role scoping across every listing route	4
Input hardening	3

Interface pass — 45 checks, 41 screenshots

The app runs in Chrome and is driven the way a person would: sign in, filter a list, open a deal, add a note, tick a task, generate a draft, paste a transcript, ask a question, review a duplicate. Each check asserts what should be on screen and captures the result.

AREA	CHECKS
Authentication	4
Dashboard	7
Deals list	4
Deal detail	9
Draft follow-up	3
Summarize meeting	1
Ask your CRM	3
Semantic search	1
Contacts	3
Tasks	1
Duplicates	2
AI usage	2
Role scoping	2
Responsive	1
Dark theme and console health	2

What the tests actually prove

Access control holds at the data layer

A member sees zero of another owner's contacts, deals, notes and tasks across every listing route, cannot read, edit, delete or re-score another owner's records, and cannot assign ownership to someone else. The admin-only routes refuse members with a 403 and anonymous callers with a 401. The same scoping is applied inside natural-language queries and semantic search, so a member cannot widen their view by asking for it.

The natural-language feature cannot be talked into a write

Five question shapes translate and run. Six rejection classes hold: unknown fields, an operator that does not fit the field type, \$-prefixed keys anywhere in the payload, over-limit requests, unsortable fields and malformed structures. A request to delete every contact is refused with a reason, and the contact count is unchanged afterwards.

An AI outage degrades rather than breaks

The entire application was exercised with no AI provider configured. Every screen renders and every feature falls back to a labelled alternative. Gateway tests separately cover a timeout, a refusal, a schema mismatch and an open circuit breaker, all of which return a result object rather than throwing into a request handler.

Untrusted text stays data

A note containing “ignore all previous instructions” is stored verbatim, flagged for a human, and never acted on. Angle brackets are neutralised so note content cannot close the wrapper that fences it, control and zero-width characters are stripped, and long input is capped.

Nothing is sent or merged without a person

Generating an email draft writes no timeline entry; only an explicit send does. A merge asks for confirmation, and cancelling it leaves the queue untouched.

Defects found and fixed

Every one of these was caught by running the thing rather than by reading it. All are fixed and covered by a test.

FOUND BY READING THE SEEDED OUTPUT

Every deal scored 100

Cause. The six score components summed past the ceiling, so the clamp flattened the top of the range and the gauge told you nothing.

Fix. Rescaled all six weights so only an exceptional deal reaches 100. Scores now spread across the full range on the seed data.

FOUND BY SEMANTIC SEARCH SCREENSHOT SHOWED EVERY NOTE THE SAME AGE

Seeded history all showed as a few minutes old

Cause. The ORM marks created timestamps immutable, so the backdating update was silently dropped. That also inflated the 30-day engagement count feeding the score.

Fix. Backdate through the raw driver. History now spans months and engagement counts are realistic.

FOUND BY API LOG DURING A RESTART

Duplicate scan crashed on a unique-index collision

Cause. Two background jobs discovering the same pair from opposite sides raced to insert it.

Fix. An idempotent upsert that treats a lost race as a no-op. The boot log went from repeated job failures to clean.

FOUND BY DRIVING THE ASK SCREEN IN A BROWSER

A rejected question showed “Unprocessable Entity”

Cause. The client read the HTTP status text instead of the structured rejection in the response body, so the user never saw the real reason.

Fix. Propagate the response body. The refusal now states why.

FOUND BY READING THE GENERATED ACTION ITEMS

Fallback meeting summaries turned speaker lines into tasks

Cause. The extractive matcher accepted any sentence containing a modal verb, so “Let’s start with security” became an action item.

Fix. Require a first-person commitment paired with an action verb, and strip speaker labels first. Five real commitments now, no noise.

FOUND BY SEMANTIC SEARCH TESTS FAILED AFTER THE OPTIMISATION

Vector reads returned corrupt data

Cause. Two bugs in the new packed-Float32 store: a lean read returns BSON Binary rather than a Buffer, and the copy could land on a non-4-byte-aligned offset, either of which breaks the Float32 view.

Fix. Accept both shapes and copy into a fresh aligned buffer.

The interface pass found no application defects

Seven checks failed on the first run. Each was investigated to root cause, and all seven were faults in the test script rather than the application.

FAILURES	ROOT CAUSE	CORRECTION
Five checks	Assertions were case-sensitive against labels the CSS uppercases. “RECORD A” and “ACTION ITEMS” were rendering correctly the whole time.	Compare case-insensitively
One check	Waited for placeholder text, which never appears in a page’s rendered text.	Wait for the input element instead
Two checks	Waited on toasts, which auto-dismiss and are inherently racy. A focused probe proved the requests returned 200 and 201 and the data saved.	Assert the outcome, not the notification
Four checks	Used a DOM-level click on tab triggers, which listen for pointer events and ignore it.	Use real mouse events

The two worth spelling out

Two checks waited on a toast notification and timed out. Rather than assume a timing quirk, a focused probe was written against the live app: it confirmed the recompute request returns 200 with its notification appearing within half a second, and the note request returns 201 with the note visible on the page. The checks were then rewritten to assert the request and the resulting change instead of a notification that dismisses itself.

A run also confirmed no uncaught console errors across the whole pass. The 5 messages captured are the deliberate 401, 403 and 422 responses the tests provoke.

Performance

Measured with `npm run bench -w @crm/api`, which loads a synthetic dataset and times the paths that grow super-linearly. Two were genuinely broken and are now fixed.

PATH	BEFORE	AFTER	CHANGE
Duplicate scan, 1,200 contacts	7,438 ms	140 ms	53× faster
Vector load, 6,000 embeddings	164 ms	36 ms	4.6× faster

Duplicate detection compared every contact against every other, which is 719,000 comparisons at 1,200 contacts and roughly 8.6 minutes of CPU at 10,000. It now generates candidates from blocking keys and scores 1.5% of the pairs. All five realistic duplicate classes are still caught, which the test suite locks in.

Vector search spent its time loading rather than comparing. Vectors are now packed Float32, normalised on write so ranking is a dot product, and cached in process.

Not verified

No live Claude call has ever run

This machine has no API key, so every check above exercised the fallback path. What is proven is the plumbing: caching, usage logging, the circuit breaker, refusal handling and every degradation path, all covered by tests using a provider double that returns canned structured output. What is not proven is the quality of model-written prose — the drafts, summaries and risk explanations. Set a key, restart, and the header pill flips from “AI offline” to the model name; the admin usage page is where you would confirm calls are landing.

Three further limits are documented in the project readme rather than fixed: deep pagination uses skip and limit, deal search by company runs an unindexed regex, and several screens poll on an interval. All three are comfortable at the scale this app is built for.

Reproducing this

COMMAND	RUNS
<code>npm test</code>	The 147 automated tests
<code>npm run typecheck</code>	All three packages
<code>npm run bench -w @crm/api</code>	The scale benchmark
<code>npm run build</code>	The production build of the web app

The interface pass needs the app running on its usual ports, then drives Chrome directly and writes its screenshots and a JSON result file.